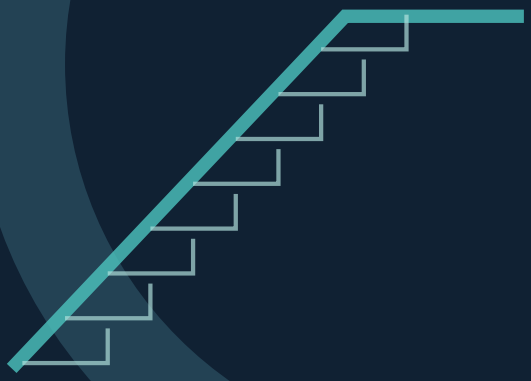
FIAP / CHALLENGE

DOMAIN DRIVEN DESIGN

SISTEMA DE MONITORAMENTO INTEGRADO DE ESCADAS ROLANTES

Modelagem de classes, orientação a objetos e documentação

RELATÓRIO TÉCNICO · JAVA



INTEGRANTES DO GRUPO

Artur Henrique Siqueira	RM566986
Davi de Souza Malta	RM560327
Guilherme Cruz Alves	RM566861
Guilherme de Oliveira Scremin	RM564788
Pedro Sales Ferreira	RM566990

ENGENHARIA DE SOFTWARE · OUTUBRO 2026

APRESENTAÇÃO DO RELATÓRIO

Sumário

Documento técnico elaborado para a etapa de Domain Driven Design (Java), integrada à disciplina de Database Design. O conteúdo descreve o recorte implementado no modelo de domínio e sua possibilidade de evolução para persistência e API.

01 Contexto do desafio e proposta 03

02 Funcionalidades e delimitação do escopo 04

03 Modelagem e diagrama de classes 05

04 Dados do domínio e relação com o banco 06

05 Quatro pilares da orientação a objetos 07

06 Javadoc e aspectos técnicos 08

07 Exemplo de uso e considerações finais 09

VISÃO RÁPIDA DO MODELO

03 classes

06 leituras de sensor

04 pilares da POO

ENTENDIMENTO DO PROBLEMA

01 Contexto e proposta

O monitoramento de escadas rolantes envolve informações de identificação, condição operacional, localização e medições dos componentes. Quando esses dados ficam separados por equipamento ou origem, torna-se mais difícil apresentar uma visão organizada do conjunto de escadas instaladas.

Problema a ser solucionado

O desafio é **estruturar e unificar os dados provenientes de escadas rolantes existentes**, preservando o vínculo entre cada equipamento, suas leituras e seu local de instalação. O foco da solução está na organização e na centralização das informações; não depende da proposta de instalar novos sensores.

Objetivo geral

Modelar, em Java, os elementos básicos de um sistema de monitoramento de escadas rolantes, com uma representação orientada a objetos compatível com a futura modelagem do banco de dados.

Objetivos específicos

- Representar cada escada por identificação, status operacional e leituras de sensores.
- Associar obrigatoriamente cada escada a uma localização cadastrada.
- Centralizar atributos comuns na classe abstrata **EquipamentoMonitorado**.
- Permitir atualização das medições e apresentação textual dos dados.
- Demonstrar abstração, encapsulamento, herança e polimorfismo em Java.

Proposta de valor: criar uma base de domínio consistente para integrar, identificar e consultar dados de múltiplas escadas. A aquisição automática de leituras, a persistência e a disponibilização por API representam etapas de evolução, não funcionalidades implementadas nas três classes avaliadas.

COMPORTAMENTOS PREVISTOS NA MODELAGEM

02 Funcionalidades e escopo

O escopo implementado é um **modelo orientado a objetos**. Ele permite criar entidades em memória, relacioná-las e exibir suas informações, sem pressupor integração ativa com equipamentos físicos.

Funcionalidades representadas pelas classes

Funcionalidade	Como é representada
Identificação de equipamentos	**EquipamentoMonitorado** possui ID, identificação e status.
Representação de escadas	**EscadaRolante** herda os dados comuns e acrescenta seis leituras.
Identificação do local	**Localizacao** descreve nome, endereço e setor de instalação.
Vinculação escada-local	O construtor de **EscadaRolante** exige um objeto **Localizacao** não nulo.
Atualização de leituras	**atualizarLeituras(...)** altera os seis valores atuais da escada.
Exibição das informações	**toString()** apresenta os atributos, com sobrescrita na subclasse.

Delimitação do estágio atual

Incluído no modelo

Entidades em memória; vínculo com localização; leitura e alteração de valores; apresentação textual; herança e polimorfismo.

Evolução posterior

Coleta automática por sensores, histórico temporal, regras de alertas, banco de dados, JDBC, API e serviços Spring.

Regra de domínio: a localização não pode ser nula durante a criação de uma escada rolante, pois o construtor utiliza **Objects.requireNonNull(...)**. A troca da referência de localização não está prevista porque o atributo foi declarado **final**.

ESTRUTURA ORIENTADA A OBJETOS

03 Modelagem de classes

O domínio foi organizado em uma classe-base abstrata e duas classes concretas. A herança reaproveita os dados dos equipamentos, enquanto a associação vincula a escada ao local de instalação.



Leitura do diagrama: EscadaRolante especializa **EquipamentoMonitorado**. Cada escada possui exatamente uma **Localizacao**; uma mesma localização pode ser compartilhada por várias escadas. Métodos e atributos estão resumidos para privilegiar a legibilidade.

COMPATIBILIDADE COM DATABASE DESIGN

04 Dados do domínio e banco

Os atributos definidos nas classes podem orientar a organização das tabelas e suas chaves. A proposta abaixo é conceitual: **o projeto apresentado ainda não implementa JDBC nem persistência de dados.**

Inventário de atributos principais

Classe	Atributos	Finalidade
EquipamentoMonitorado	id, identificacao, status	Dados comuns a qualquer equipamento monitorado.
EscadaRolante	vibracao, temperatura, corrente, tensao, nivelOleo, velocidade	Valores atuais das seis grandezas monitoradas.
EscadaRolante	localizacao: Localizacao	Associação obrigatória ao local de instalação.
Localizacao	id, nome, endereco, setor	Informações cadastrais e físicas da localização.

Proposta de tradução para o modelo relacional

TB_LOCALIZACAO

PK: id
Campos: nome, endereco, setor
Relacionamento: um local pode conter várias escadas.

TB_ESCADA_ROLANTE

PK: id
FK: localizacao_id
Campos: identificacao, status e leituras de sensores.

Uma estratégia simples é incorporar os campos herdados de **EquipamentoMonitorado** na tabela de escadas, dispensando uma tabela exclusiva para a classe abstrata. A chave estrangeira **localizacao_id** representa o vínculo entre escada e localização.

Observação sobre as leituras

As seis medições são armazenadas como **double** na implementação atual, sem unidade de medida explícita no nome do atributo. Para integração com o dataset e com o banco, as unidades e a frequência das leituras devem ser confirmadas. Uma futura tabela de histórico poderá registrar data, hora e múltiplas medições por equipamento.

FUNDAMENTAÇÃO TÉCNICA

05 Pilares da orientação a objetos

O desenho das classes utiliza os quatro pilares da programação orientada a objetos, diretamente identificáveis na implementação.

01 ABSTRAÇÃO

EquipamentoMonitorado é uma classe abstrata. Ela agrupa atributos comuns e define o contrato getTipoEquipamento(), cuja implementação fica a cargo de cada equipamento concreto.

```
abstract class EquipamentoMonitorado
```

02 ENCAPSULAMENTO

Os atributos das três classes são privados. A leitura ocorre por getters e a alteração autorizada por métodos como setStatus() e atualizarLeituras().

```
private double temperatura;
```

03 HERANÇA

EscadaRolante estende EquipamentoMonitorado e reutiliza identificação, status e seus comportamentos sem duplicar a modelagem dos campos comuns.

```
class EscadaRolante extends EquipamentoMonitorado
```

04 POLIMORFISMO

EscadaRolante sobrescreve getTipoEquipamento() e toString(). Uma referência EquipamentoMonitorado pode apontar para uma escada e executar o comportamento concreto.

```
EquipamentoMonitorado item = escada;
```

ORGANIZAÇÃO DA DOCUMENTAÇÃO

06 Javadoc e critérios técnicos

As classes fornecidas utilizam comentários Javadoc para descrever suas responsabilidades, atributos, construtores, métodos e retornos. A documentação também facilita a avaliação da API pública do modelo.

Elementos da documentação

Marcador	Aplicação no projeto
@author	Identificação da autoria das classes.
@version	Versão da documentação/classe (1.0).
@param	Descrição dos argumentos dos construtores e métodos.
@return	Descrição dos resultados dos getters e de toString().
@see	Referências entre classes relacionadas; usadas em EscadaRolante.
@since	Indicação da versão inicial; recomendada para completar a padronização.
@throws	Documentação da exceção quando uma localização nula é fornecida.

Exemplo de documentação do construtor

```
/**
 * Cria uma escada rolante com local obrigatório.
 * @param localizacao local da instalação
 * @throws NullPointerException se localizacao for nula
 * @since 1.0
 */
```

Ponto de atenção: no código apresentado, **@since** ainda não foi incluído e o construtor de **EscadaRolante** ainda não documenta com **@throws** a validação feita por Objects.requireNonNull(). Esses dois complementos são recomendados antes da entrega definitiva do projeto ZIP.

DEMONSTRAÇÃO E CONCLUSÃO

07 Uso e considerações finais

O enunciado exige que as classes concretas sejam instanciadas na classe Main e tenham seus dados impressos por toString() ou método equivalente. A classe abstrata não pode ser instanciada diretamente, mas pode ser utilizada como tipo de referência.

Exemplo ilustrativo de utilização em Main

```
Localizacao local = new Localizacao(
    1, "Estação Exemplo", "Endereço ilustrativo", "Acesso A");

EscadaRolante escada = new EscadaRolante(
    1, "ESC-001", "Operacional",
    2.5, 35.0, 10.0, 220.0, 80.0, 0.5, local);

EquipamentoMonitorado equipamento = escada;
System.out.println(local);
System.out.println(equipamento);

escada.atualizarLeituras(2.7, 36.0, 10.5, 220.0, 79.0, 0.5);
System.out.println(escada);
```

Os nomes, valores e endereços acima são apenas exemplos didáticos; não representam registros reais do dataset. O trecho ilustra como as instâncias concretas e o polimorfismo podem ser demonstrados na aplicação.

Conclusão

O modelo construído estabelece uma base coerente para organizar dados de escadas rolantes, aplicar os quatro pilares da orientação a objetos e manter a relação obrigatória com sua localização. A abordagem valoriza a **centralização dos dados já disponíveis** e oferece um ponto de partida para evolução do sistema com persistência, histórico de leituras e API.

Entregável acadêmico: este relatório acompanha a pasta do projeto Java em um único ZIP, conforme solicitado na atividade. Os arquivos-fonte e a classe Main devem ser conferidos separadamente antes do envio.